Communication Protocol Effects on Multi-Agent System Efficiency and Task Completion

STAT GR5293 Final Project Presentation

Yi Zhang Xian Zhang Tianyu Zhan

Department of Statistics, Columbia University

Spring 2026

Presentation Roadmap

Problem Statement	Major Contributions	Evaluation
Why communication format is an uncontrolled design choice in multi-agent LLM systems.	Controlled protocol comparison, reusable pipeline, statistical analysis, and a live demo.	360 pipeline runs across four protocols, three domains, and multiple efficiency and quality metrics.

Rubric alignment: the presentation is organized around the three 10% final-presentation categories: problem statement, contributions, and evaluation.

Problem Statement

Multi-agent LLM systems are increasingly built as pipelines: agents plan, execute, critique, retrieve, or integrate intermediate work.

Most frameworks expose inter-agent communication as a flexible engineering choice, but do not answer a basic empirical question:

Core Question

How does the communication protocol between agents change token cost, latency, and task completion quality?

- The protocol can be plain natural language, Markdown, JSON, or a shared blackboard.
- The same protocol may not be optimal for math, reading comprehension, and news analysis.
- The practical target audience is developers choosing protocols in LangGraph, AutoGen, CrewAI, or custom agent systems.

Research Questions and Hypotheses

	Question	Hypothesis
RQ1	How much does protocol choice affect efficiency?	Shared memory costs the most tokens; structured formats should reduce ambiguity.
RQ2	How much does protocol choice affect task completion?	Protocol effects may vary by domain and evaluator.
RQ3	Is there a protocol-by-domain interaction?	The best protocol should change across math, reading, and news.

Scope boundary: the project studies the protocol layer, not a new task-specific agent application.

System Model

Fixed three-agent chain

1. Planning Agent: decomposes the task.
2. Execution Agent: completes subtasks.
3. Integration Agent: produces the final answer.

Held constant across all conditions:

- model: `gpt-4o-mini`
- temperature: 0.3
- agent roles and system prompts
- max tokens by domain

Manipulated factor: protocol

- `NL`: explicit plain-prose instruction.
- `MARKDOWN`: headings and bullet structure.
- `JSON`: valid JSON object with parse validation and one retry.
- `SHARED_MEMORY`: global blackboard; every agent reads the full state snapshot.

Design Principle

Change the message format, not the agent architecture.

Experimental Design

Factorial grid

- 4 protocols: `NL`, `MARKDOWN`, `JSON`, `SHARED_MEMORY`
- 3 domains: GSM8K math, SQuAD reading, curated news
- 10 samples per domain
- 3 repetitions per cell

$$4 \times 3 \times 10 \times 3 = 360 \text{ pipeline runs}$$

Metrics

- Efficiency: prompt tokens, completion tokens, total tokens.
- Runtime: wall-clock latency.
- Quality: numeric exact match for math, SQuAD token-F1 for reading, mean ROUGE-2/ROUGE-L F1 for news.

Inference

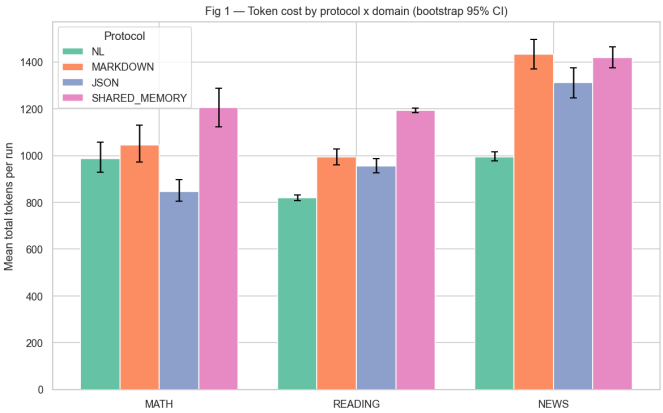
- Two-way ANOVA, Tukey HSD, Cohen's d.
- 2,000-resample bootstrap confidence intervals.

Contribution 1: Controlled Protocol Benchmark

- A reusable experiment harness separates the protocol factor from agent roles and task data.
- Every inter-agent message is logged with sender, receiver, content, token split, latency, finish reason, and parse-error status.
- The resulting dataset contains 360 run summaries and 1,080 message records.

Artifact	File	Purpose	Rubric Value
Pipeline	`pipeline.py`	reproducible experiment runner	technical depth
Results	`results/*.csv/jsonl`	quantitative evidence	evaluation
Figures	`figures/*.png`	presentation/report visuals	clarity
Demo	`app.py`	end-to-end protocol selector	practical impact

Result: Protocol Strongly Affects Token Cost

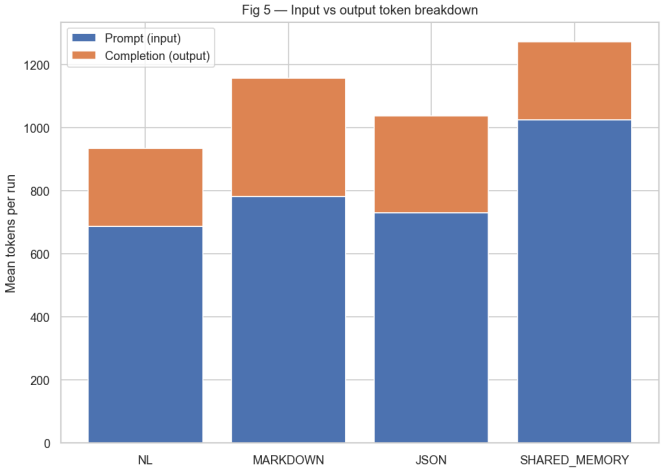


Two-way ANOVA on total tokens

Term	F	p	η^2
Protocol	87.40	$< .001$	.262
Domain	148.19	$< .001$	.296
Protocol $\times$ domain	15.66	$< .001$	.094

Verdict: Shared memory is the most expensive protocol in every domain, averaging $+338$ tokens per run versus `NL`.

Why the Cost Changes

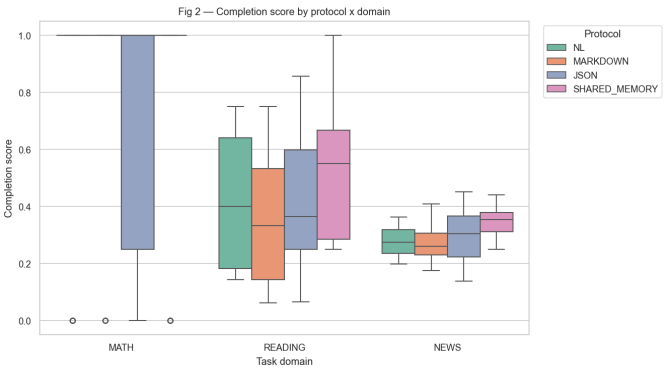


- `SHARED_MEMORY` inflates prompt tokens because downstream agents receive the full blackboard snapshot.
- `MARKDOWN` often increases completion tokens because headings and bullets add verbosity.
- `JSON` is cheapest on math, but not pooled-cheapest overall.
- `NL` is a strong budget default for reading and news.

Efficiency Ordering

Pooled token cost: `NL` < `JSON` < `MARKDOWN` < `SHARED_MEMORY`

Result: Quality Effects Are Domain-Dependent

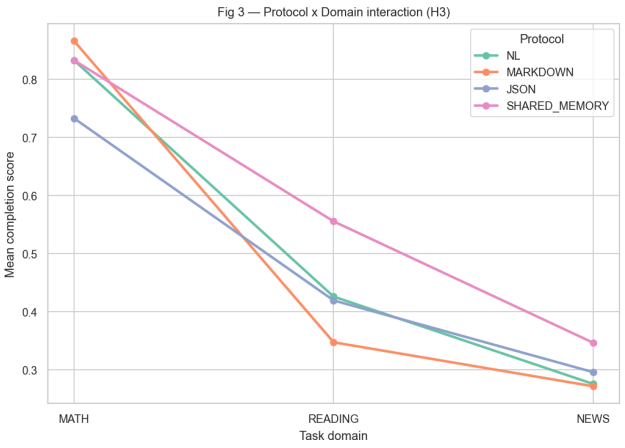


Per-domain ANOVA on completion

Domain	F	p	η^2	Best
Math	0.66	.58	.017	Markdown
Reading	4.39	.006	.102	Shared mem.
News	9.51	$< .001$	.197	Shared mem.

Verdict: protocol choice is mostly a cost lever for math, but a quality lever for reading and news.

Interaction Pattern

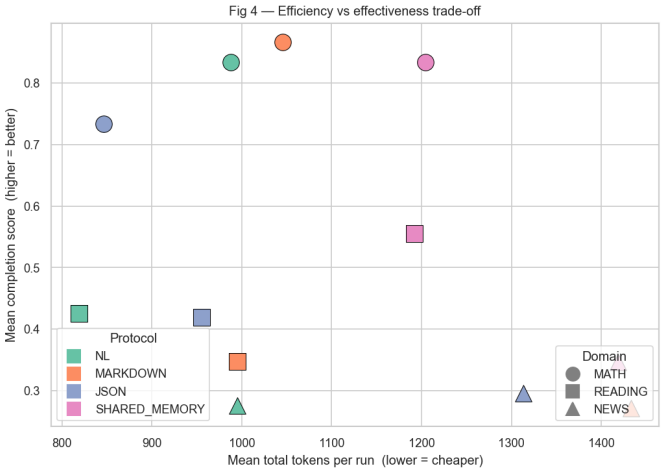


- The best protocol changes by domain.
- Math: protocols cluster closely; Markdown leads by a small margin.
- Reading: shared memory preserves passage context and improves token-F1.
- News: shared memory preserves facts and figures better.

Statistical Note

Pooled completion interaction: $p = .203$; per-domain effect sizes range from .017 to .197.

Efficiency–Effectiveness Trade-off



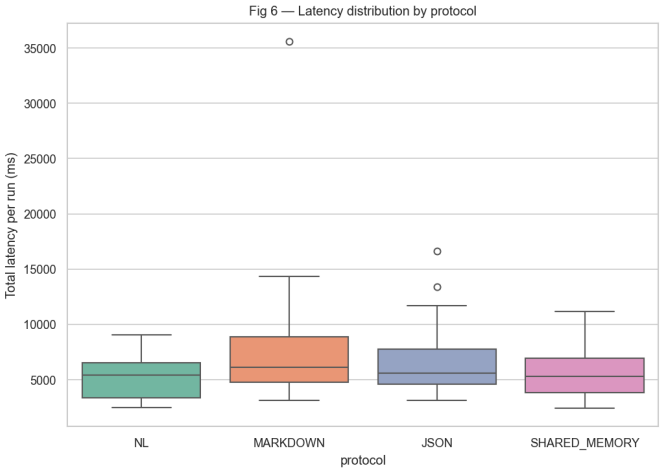
Practitioner recommendation

- Math: choose `MARKDOWN` for best completion, `JSON` for lowest cost.
- Reading: choose `SHARED_MEMORY` for quality; `NL` for budget.
- News: choose `SHARED_MEMORY` when factual completeness matters.

Main Trade-off

Shared memory buys quality on open-text tasks, but it is never the cheapest option.

Latency and Operational Cost



Latency ANOVA

- Protocol: $F = 16.55$, $p < .001$, $\eta^2 = .070$.
- Domain: $F = 136.44$, $p < .001$, $\eta^2 = .385$.
- Interaction: $F = 6.38$, $p < .001$, $\eta^2 = .054$.

Interpretation

- Domain complexity dominates latency.
- Protocol still has a detectable operational effect.
- Token-efficient protocols also tend to reduce estimated API cost.

Project Demo

Streamlit workflow

1. User enters a free-text task.
2. LLM classifier predicts domain.
3. App selects the empirically best protocol from `results_summary.csv`.
4. Planner, executor, and integrator run.
5. Dashboard reports protocol, tokens, latency, cost, and final answer.

Demo Contribution

The demo converts the offline benchmark into an actionable protocol-selection tool.

Implementation

- Single-page Streamlit app in `app.py`.
- Auto-selection can be overridden manually.
- Raw message log exposes every inter-agent exchange for debugging.

Limitations and Future Work

Limitations

- Sample size: 10 tasks per domain and 3 repetitions per cell.
- Single base model: all agents use `gpt-4o-mini`.
- Three-agent sequential chain only; no branching or debate.
- Completion interaction on quality is underpowered in the pooled test.

Next steps

- Replicate with larger domain samples.
- Add XML, YAML, and hybrid protocols.
- Test heterogeneous agent models by role.
- Switch protocols dynamically based on confidence or token budget.

Takeaways

1. Communication protocol has a large efficiency effect: protocol explains $\eta^2 = .262$ of token variance.
2. Shared memory is consistently expensive, but improves completion on reading and news.
3. Math is relatively insensitive to protocol quality; protocol choice there is mainly a cost decision.
4. The project provides reusable artifacts: `pipeline.py`, 360-run results, figures, and a Streamlit demo.

Final Recommendation

Do not choose a communication protocol globally. Choose it by domain and by whether the immediate priority is cost or completion quality.

Backup: Cell-Level Summary

Protocol	Domain	Tokens	Prompt	Completion	Score
NL	Math	988	665	323	.833
NL	Reading	819	717	102	.426
NL	News	995	680	315	.275
MARKDOWN	Math	1046	695	351	.867
MARKDOWN	Reading	995	794	201	.347
MARKDOWN	News	1433	860	574	.271
JSON	Math	847	600	247	.733
JSON	Reading	955	777	178	.419
JSON	News	1313	815	498	.296
SHARED_MEMORY	Math	1204	931	273	.833
SHARED_MEMORY	Reading	1192	1095	97	.556
SHARED_MEMORY	News	1418	1052	366	.346

Source: `results/results_summary.csv`; each cell has $N = 30$ runs.